## DAY 4 — Data Cleaning, Preprocessing & Handling Missing Data

Day 4 focused on a very important part of the Data Science pipeline — **data preprocessing**. It is often said that **80% of the data scientist's time goes into cleaning and preparing data**, and today I understood why. Raw data is never perfect; it includes missing values, inconsistencies, unwanted columns, wrong formats, duplicates, and noise. Preprocessing transforms messy data into meaningful, usable information.

### 🔹 Understanding the Need for Data Preprocessing

We began by identifying why preprocessing is essential:

- To remove errors and inconsistencies
- To improve model accuracy
- To handle missing or corrupted values
- To bring data in a uniform structure
- To prepare features for machine learning algorithms

A dataset without cleaning would lead to incorrect predictions and misleading conclusions.

---

### 🔸 1. Handling Missing Values (Core Topic of the Day)

Missing data is a very common issue. We studied three major methods:

**A. Deleting missing data**

- Remove rows containing nulls
- Remove columns with too many missing values
- Use cautiously (risk of losing important information)

**B. Imputation (Filling missing values)**

- Fill with **mean**, **median**, or **mode**
- Forward fill / backward fill for time-series
- Constant value filling (e.g., 0 or "Unknown")

**C. Advanced Methods**

- Using algorithms like **KNN Imputer**
- Predicting missing values using ML models

We also practiced commands like:

```
df.isnull().sum()
df.fillna()
```

df.dropna()

---

### 🔸 2. Handling Duplicates

Duplicate rows can distort analysis.
I learned:

- df.duplicated()
- df.drop_duplicates()

Duplicates are especially harmful in financial datasets like stock prices.

---

### 🔸 3. Encoding Categorical Variables

Machine learning models only understand numbers, not text.

We studied two encoding types:

**1. Label Encoding**

Converts categories into integers
Example: Male → 0, Female → 1

**2. One-Hot Encoding**

Creates separate binary columns
Example:
Red → [1,0,0]
Blue → [0,1,0]
Green → [0,0,1]

Using:

pd.get_dummies()

---

### 🔸 4. Feature Scaling

Scaling improves model performance.
Two major types:

**Standardization**

(Z-score scaling)

x = (x - mean) / std

**Normalization**

(Min-Max scaling)

x = (x - min) / (max - min)

---

**Key Takeaways from Day 4**

- Clean data leads to accurate models
- Handling missing values is crucial
- Encoding and scaling prepare data for ML
- Preprocessing determines the performance of the entire project

This day helped me become more confident working with real-world datasets.